SDS – ESPECIFICACION DE DISEÑO DE SOFTWARE

1. Generalidades del Documento

1.1 Objetivo del Documento

Definir de manera detallada la arquitectura, estructura, componentes, interfaces y modelo de datos del sistema, estableciendo las decisiones técnicas necesarias para transformar los requerimientos funcionales y no funcionales en una solución de diseño coherente, trazable y viable

1.2 Audiencia del Documento

Este documento está dirigido a:

- Equipo de desarrollo
- Arquitecto de software
- Equipo de pruebas
- Stakeholders técnicos

1.3 Acrónimos

Acrónimo	Significado
SDS	Software Design Specification
UML	Unified Modeling Language
API	Application Programming Interface

Acrónimo	Significado
BD	Base de Datos
ER	Entidad-Relación

2. Introducción

2.1 Descripción del Problema

En la actualidad, los conductores no cuentan con una herramienta centralizada que les permita consultar de manera rápida y confiable los precios del combustible, comparar estaciones de servicio cercanas, calcular rutas óptimas y estimar el costo total de un viaje.

Esta falta de información integrada dificulta la toma de decisiones eficientes, genera mayores gastos en combustible y reduce la posibilidad de optimizar tiempos y recursos.

2.2 Objetivos de la Aplicación

Definir una aplicación que permita a los usuarios consultar estaciones de servicio, comparar precios en tiempo real, buscar rutas óptimas, simular costos de viaje y calcular ahorro estimado, facilitando la toma de decisiones eficientes y económicas en el consumo de combustible

2.3 Restricciones

- Compatibilidad con dispositivos
- Limitaciones en acceso a datos reales de precios en tiempo real
- La infraestructura en la nube debe optimizarse para evitar sobrecostos en almacenamiento
- El desarrollo debe realizarse con los recursos tecnológicos propios del equipo
- Acceso limitado a servicios externos o APIs comerciales.

3. Arquitectura de Software

3.1 Estilo de Arquitectura

estilo seleccionado:

Se utilizará arquitectura Monolítica

Justificación de la elección:

- **Desarrollo Unificado:** Todo el código se gestiona en un único lugar, facilitando el desarrollo inicial.
- **Alto Rendimiento:** Al ser componentes locales, la comunicación interna es más rápida que en sistemas distribuidos.
- **Sencillez:** Menor sobrecarga cognitiva en la gestión del código y la implementación.
- **Despliegue Simple:** Se publica la aplicación como una sola unidad.

3.2 Vistas de Arquitectura

3.2.1 Vista de Contexto (UML – Casos de Uso)

- Actores

Usuario: persona que interactúa con el sistema, para registrarse e iniciar sesión.

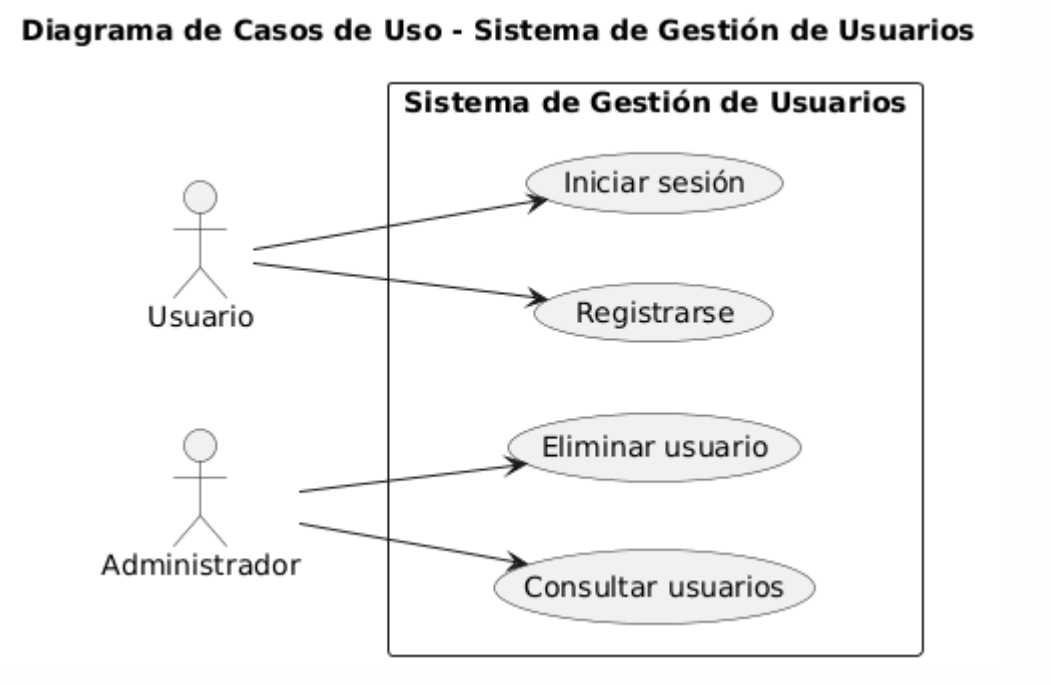
Administración : aquel usuario con privilegios para a acceder consultar y eliminar registros del sistema.

- Casos de uso principales

- Registro de usuarios

- Iniciar sesión
- Consultar usuario
- Eliminar usuario

- Diagrama UML



- Descripción breve de cada caso de uso

Registrarse
Permite al usuario ingresar sus datos personales (nombre, correo, contraseña) para crear una cuenta en el sistema.

Iniciar sesión
Permite al usuario autenticarse utilizando su correo y contraseña previamente registrados.

Consultar usuarios
Permite al administrador visualizar los usuarios almacenados en la base de datos.

Eliminar usuario
Permite al administrador eliminar un usuario existente mediante su ID.

3.2.2 Vista Funcional (UML – Componentes)

Componentes del sistema

En la arquitectura monolítica, el sistema está compuesto por los siguientes módulos internos:

- **Módulo de Presentación**
- **Módulo de Lógica de Negocio**
- **Módulo de Acceso a Datos**
- **Base de Datos SQLite**

Todos estos módulos forman parte de una única aplicación desplegada en el servidor.

Relaciones entre componentes

- El Módulo de Presentación envía solicitudes al Módulo de Lógica.
- El Módulo de Lógica procesa reglas del sistema y realiza validaciones.
- El Módulo de Lógica solicita operaciones al Módulo de Acceso a Datos.
- El Módulo de Acceso a Datos se comunica directamente con la Base de Datos SQLite.

Responsabilidades de cada componente

Módulo de Presentación

- Capturar información del usuario.
- Mostrar resultados y mensajes.
- Gestionar formularios web.

Módulo de Lógica de Negocio

- Validar datos.
- Aplicar reglas del sistema.
- Gestionar autenticación.
- Controlar operaciones CRUD.

Módulo de Acceso a Datos

- Ejecutar consultas SQL.
- Gestionar conexión a la base de datos.
- Garantizar integridad de datos.

Base de Datos

- Almacenar información persistente.
- Garantizar restricciones y relaciones.

Diagrama de componentes

3.2.3 Vista de Información (Modelo Entidad-Relación)

- Entidades

  Usuario, estaciones de servicio y administradores

- Atributos

Usuario: Id_Usuario, nombre, dirección, teléfono

Estación de servicio: Id_estación, nombre, dirección, numero de empleado, cantidad de gasolina

Administrador: Id_Admin, nombre, dirección, teléfono

- Relaciones

  -El administrador maneja los usuarios

  -Los usuarios buscan las estaciones de servicio

  -El administrador maneja las estaciones de servicio

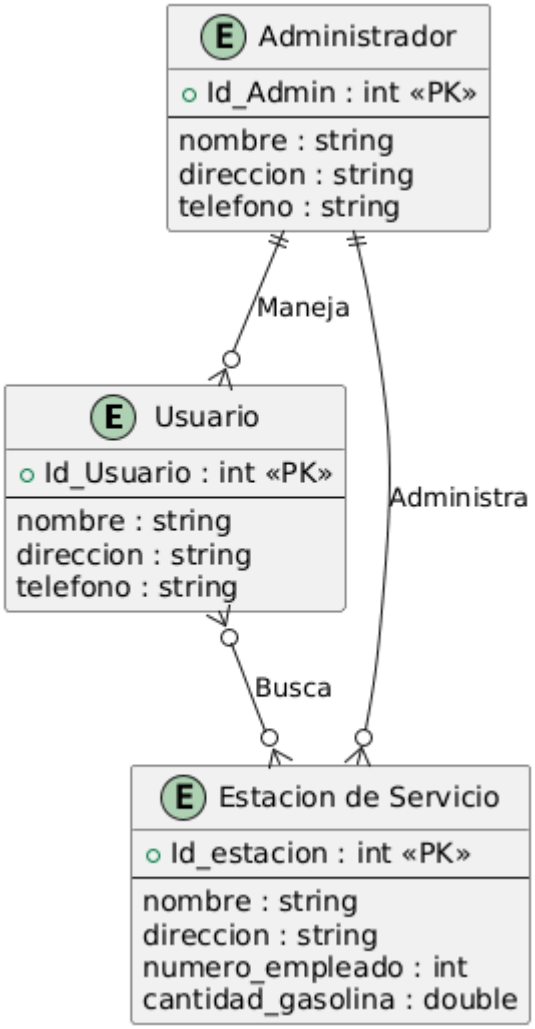
- Cardinalidades

  -Administrador 1:N Usuario

  -Administrador 1:N Estación

  -Usuario N:M Estación

- Diagrama ER



3.2.4 Vista de Despliegue (UML – Despliegue)

- Nodos físicos o virtuales

  -Cliente (físico): dispositivo móvil

  -Servidor de aplicaciones: gestión de usuarios, gestión de estaciones, gestión de inventario, etc.

  -Servidor de bases de datos: base de datos relacional, tablas de usuarios, etc.

- Servidores:

  -Servidor de aplicaciones: Ejecuta la lógica del sistema

  -Servidor de bases de datos: guarda la información y permite consultas.

- Bases de datos

  -Base relacional principal: maneja la información de todas las entidades importantes.

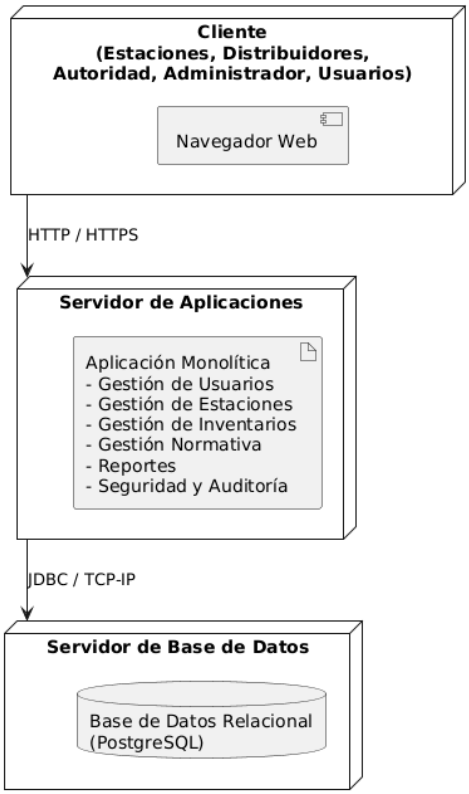
- Comunicación entre nodos

  -Cliente envía una solicitud HTTP y servidor de aplicaciones la completa.

  -Servidor de aplicaciones consulta base de datos y servidor de bases de datos devuelve la información.

  -servidor de aplicaciones responde al cliente

- Diagrama de despliegue



3.2.5 Vista de Desarrollo (UML – Paquetes)

- Organización del código

  -Presentación (interfaz y controladores)
  -servicio (lógica de negocio)
  -repositorio (acceso a datos)
  -modelo (entidades del sistema)
  -seguridad (seguridad y control de acceso)

- Paquetes o módulos

  -Presentación: solicitudes rest

  -servicio: lógica de negocio, gestiones y control de transacciones

  -Repositorio: operaciones CRUD, consulta en bases de datos y interfaces

-Modelo: entidades y sus respectivos comportamientos

-Seguridad: sistema de autenticación, gestión de roles, etc.

- Dependencias entre paquetes

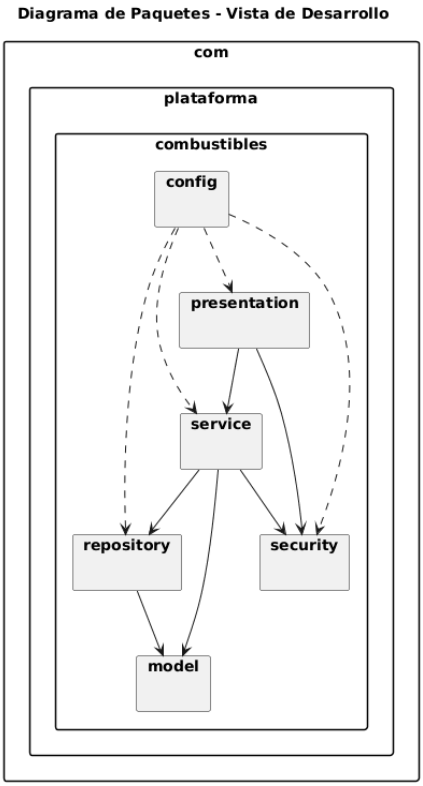
-Presentación depende de servicio

-Servicio depende de repositorio y modelo

-Repositorio depende de modelo

-Seguridad lo utiliza presentación y servicio

- Diagrama de paquetes



3.2.6 Vista de Concurrencia (UML – Procesos)

- Procesos principales

  -Proceso principal del servidor: servidor web con aplicación monolitica y conexión a base de datos

  -Procesos externos: Cliente y base de datos

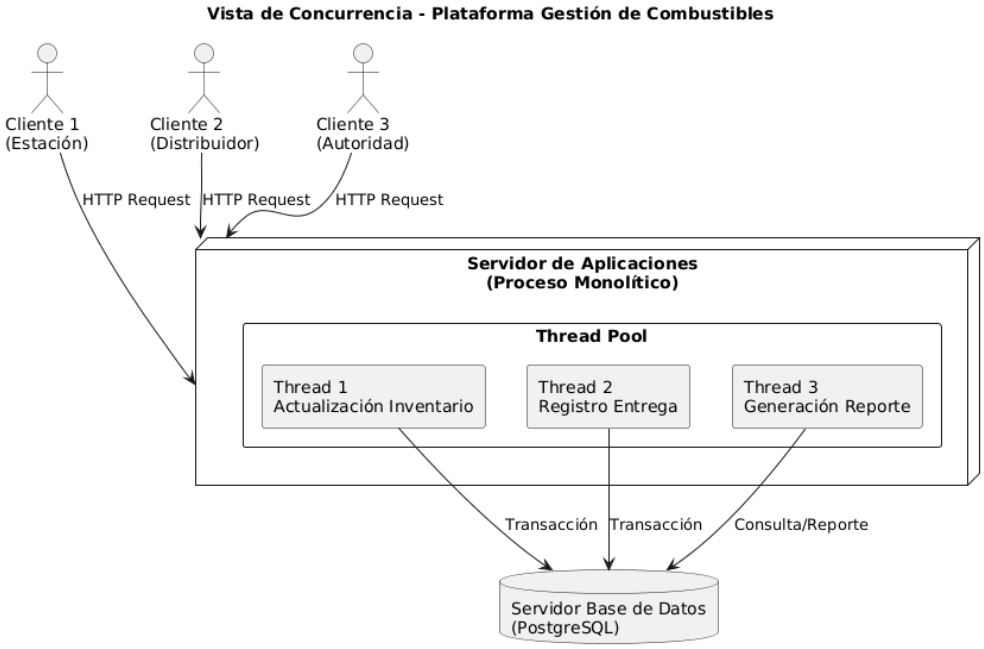
- Manejo de concurrencia

  -Concurrencia por hilos: las solicitudes se ejecutan en hilos diferentes

  -Concurrencia en bases de datos: bloqueo optimista y pesimista

- Hilos si aplica
  -Hilos de peticiones

- Diagrama de procesos



3.2.7 Vista Operacional

- Flujo operativo del sistema

El flujo operativo del sistema DieselMaps se desarrolla de la siguiente manera:

1. El usuario accede al sistema desde un navegador web.
2. El servidor recibe la solicitud HTTP.
3. El sistema valida si el usuario está autenticado.
4. El usuario puede:
    a. Registrarse
    b. Iniciar sesión
    c. Consultar estaciones
    d. Simular un viaje
5. La lógica de negocio procesa la solicitud.
6. Se consulta o actualiza la base de datos SQLite.
7. El servidor genera una respuesta.
8. El cliente recibe y visualiza la información.

- En el caso del administrador:

1. Inicia sesión con rol administrador.
2. Accede al panel de gestión.
3. Puede registrar, editar o eliminar estaciones.
4. Los cambios se almacenan en la base de datos.

- **Ciclo de vida del sistema**

El flujo operativo del sistema DieselMaps se desarrolla de la siguiente manera:

1. El usuario accede al sistema desde un navegador web.
2. El servidor recibe la solicitud HTTP.
3. El sistema valida si el usuario está autenticado.
4. El usuario puede:
    a. Registrarse
    b. Iniciar sesión
    c. Consultar estaciones
    d. Simular un viaje
5. La lógica de negocio procesa la solicitud.
6. Se consulta o actualiza la base de datos SQLite.
7. El servidor genera una respuesta.
8. El cliente recibe y visualiza la información.

En el caso del administrador:

1. Inicia sesión con rol administrador.
2. Accede al panel de gestión.
3. Puede registrar, editar o eliminar estaciones.
4. Los cambios se almacenan en la base de datos.

- **Logs y monitoreo**

El sistema implementa registro de eventos (logs) para garantizar control y trazabilidad.

Se registran:

- Intentos de inicio de sesión.
- Registro de nuevos usuarios.
- Eliminación de estaciones.
- Actualización de precios.
- Errores del sistema.

Los logs permiten:

- Auditoría de acciones administrativas.
- Identificación de fallos.
- Seguimiento de actividad sospechosa.

El monitoreo incluye:

- Control de disponibilidad del servidor.
- Verificación de conexión con base de datos.
- Supervisión de errores críticos.

- Gestión de errores

El sistema implementa registro de eventos (logs) para garantizar control y trazabilidad.

Se registran:

- Intentos de inicio de sesión.

- Registro de nuevos usuarios.
- Eliminación de estaciones.
- Actualización de precios.
- Errores del sistema.

Los logs permiten:

- Auditoría de acciones administrativas.
- Identificación de fallos.
- Seguimiento de actividad sospechosa.

El monitoreo incluye:

- Control de disponibilidad del servidor.
- Verificación de conexión con base de datos.
- Supervisión de errores críticos.

3.3 Decisiones de Arquitectura

Documentar decisiones importantes:

Decisión	justificación	Impacto
Arquitectura monolítica	Se selecciona por simplicidad, desarrollo, facilidad de implementación y menor complejidad para un proyecto académico	Despliegue sencillo y mantenimiento centralizado. Limitada escalabilidad en sistemas grandes
Arquitectura microservicio	No se adaptó debido a la complejidad de configuración y comunicación entre servicios	Reduce complejidad inicial, pero, limita escalabilidad independiente por modulo
Arquitectura cliente-servidor	Permite separar el cliente del servidor donde se ejecuta la lógica y la base de datos	Mejora organizacion del sistema y permite acceso remoto mediante HTTP
Arquitectura MVC	Facilita la separacion entre modelo,vista y controlador dentro del monolito	Mejora mantenibilidad, organización del codigo y claridad estructural
Arquitectura	Base de datos ligera y facil de integrar en proyectos	Implementación rapida y sencilla, pero no adecuada para alta concurrencia

4. Diseño Detallado

4.1 Especificación de la Interfaz de Usuario

- Mockups o prototipos
- El sistema DieselMaps contará con las siguientes pantallas principales:

    1. Pantalla de Registro.
    2. Pantalla de Inicio de Sesión.
    3. Pantalla Principal (Mapa interactivo).
    4. Pantalla de Consulta de Ubicación.

5. Panel de Administrador.
6. Pantalla de Simulación de Costo de Viaje.

- Descripción de cada pantalla

1. Pantalla de Registro de Usuario

Descripción:

Esta pantalla permite a los nuevos usuarios crear una cuenta dentro del sistema DieselMaps. Contiene un formulario con los siguientes campos:

- Nombre completo
- Correo electrónico
- Contraseña
- Confirmación de contraseña

Incluye un botón “Registrarse” y un enlace para ir a la pantalla de inicio de sesión.

Funcionalidad:

- Validación de formato de correo electrónico.
- Verificación de coincidencia de contraseñas.
- Mensajes de error si los datos no cumplen las reglas.
- Confirmación exitosa cuando el registro se completa correctamente.

2. Pantalla de Inicio de Sesión

Descripción:

Permite a los usuarios autenticarse en el sistema mediante:

- Correo electrónico
- Contraseña

Incluye botón “Iniciar Sesión” y enlace a “Registrarse”.

Funcionalidad:

- Validación de credenciales contra la base de datos.
- Redirección al mapa principal si las credenciales son correctas.

- Mensaje de error en caso de datos incorrectos.
- Diferenciación entre usuario normal y administrador.

3. Pantalla Principal – Mapa Interactivo

Descripción:

Es la pantalla central del sistema. Contiene:

- Mapa interactivo (Google Maps o API similar).
- Barra de búsqueda.
- Botón para consultar estaciones cercanas.
- Filtros avanzados (precio, marca, tipo de combustible).
- Menú de navegación superior.

Funcionalidad:

- Visualización de ubicaciones registradas.
- Mostrar marcadores de estaciones de combustible.
- Mostrar detalles al hacer clic en un marcador.
- Geolocalización en tiempo real del usuario.

4. Pantalla de Consulta de Ubicación

Descripción:

Muestra la información detallada de una estación seleccionada.

Información mostrada:

- Nombre de la estación
- Dirección
- Precio del combustible
- Marca
- Ubicación en el mapa
- Botón para calcular ruta
- Botón para simular costo de viaje

Funcionalidad:

- Mostrar detalles almacenados en base de datos.
- Calcular distancia desde la ubicación actual.
- Permitir reportar actualización de precios (usuario registrado).

5. Pantalla de Simulación de Costo de Viaje

Descripción:

Permite calcular el costo estimado de un viaje según:

- Punto de origen
- Punto de destino
- Consumo del vehículo (km/l)
- Precio del combustible

Incluye botón “Calcular”.

Funcionalidad:

- Cálculo automático del consumo estimado.
- Mostrar costo total del viaje.
- Mostrar distancia total.
- Tiempo estimado de recorrido.

6. Panel de Administrador

Descripción:

Pantalla exclusiva para usuarios con rol administrador.

Permite:

- Registrar nuevas ubicaciones.
- Editar estaciones existentes.
- Eliminar estaciones.
- Gestionar usuarios.
- Actualizar precios manualmente.

Incluye:

- Tabla de registros.
- Botones de editar y eliminar.
- Formularios de registro de estación.

Funcionalidad:

- Validación de permisos.
- Control total sobre la información almacenada.
- Registro de cambios en logs del sistema.

- **Flujo de Navegación**

1. Usuario ingresa al sistema.
2. Se registra o inicia sesión.
3. Accede al mapa principal.
4. Busca estación.
5. Consulta detalles.
6. Puede simular costo o calcular ruta.
7. Si es administrador, accede al panel de gestión.

- **Reglas de Validación**

- Correo con formato válido.
- Contraseña mínimo 8 caracteres.
- Solo administradores pueden registrar estaciones.
- Campos obligatorios no pueden quedar vacíos.
- Precios deben ser valores numéricos positivos.

- **Diseño**

El sistema será adaptable a:

- Computadores.
- Tablets.
- Dispositivos móviles.

4.2 Modelo de Datos (Entidad-Relación)

- **Descripción formal del modelo**

El sistema DieselMaps utiliza un modelo de base de datos relacional implementado en SQLite.
El modelo permite almacenar información de usuarios, estaciones de servicio, reportes de precios y simulaciones de viaje.

El diseño garantiza:

- Integridad referencial
- Consistencia de datos
- Eliminación de redundancias
- Seguridad en la autenticación

El sistema sigue una estructura normalizada para optimizar almacenamiento y consultas.

- **Tablas**

Usuario

Campo	Tipo	Descripción

Id_usuario	Integer	Identificador único
nombre	text	Nombre completo
correo	text	Correo electrónico
contraseña	text	Contraseña cifrada
rol	text	usuario/administrador

Estación

Campo	Tipo	Descripción
Id_estacion	Integer	Identificador único
nombre	text	Nombre de estación
Dirección	text	Dirección de estación
latitud	real	Coordenada geográfica
Longitud	real	Coordenada geográfica
Marca	text	Marca combustible

Precio Combustible

Campo	Tipo	Descripción
Id_precio	Integer	Identificador
Tipo de combustible	Text	Diesel, Corriente, Extra
precio	Real	Valor por litro
Fecha actualización	Date	Fecha actual de combustible
Id_estacion	Integer	FK estación

Simulación de viaje

Campo	Tipo	Descripción
Id_simulacion	Integer	Identificador
Origen	Text	Punto inicial
Destino	Text	Punto final
Distancia por km	real	Distancia calculada
Costo_estimado	real	Costo total
Id_usuario	Integer	FK usuario

- Llaves primarias (pk)

  - Usuario → id_usuario.
  - Estación → id_estacion
  - PrecioCombustible → id_precio
  - SimulacionViaje → id_simulacion

- Llaves foráneas

- PrecioCombustible.id_estacion → Estacion.id_estacion
- SimulacionViaje.id_usuario → Usuario.id_usuario

Estas relaciones permiten:

- Una estación puede tener varios registros de precio.
- Un usuario puede tener múltiples simulaciones.

- Restricciones

- correo → UNIQUE
- contraseña → NOT NULL
- precio → CHECK (precio > 0)
- rol → CHECK (rol IN ('usuario','administrador'))
- FK con restricción ON DELETE CASCADE

- Normalización

- Primera Forma Normal (1FN)

No existen campos multivaluados.

- Segunda Forma Normal (2FN)

Todos los atributos dependen completamente de su clave primaria.

- Tercera Forma Normal (3FN)

No existen dependencias transitivas.

Esto evita redundancia y mejora eficiencia.

4.3 Modelo de Clases (UML)

Clases Principales

- **Clase Usuario**

Atributos

- id_usuario : int
- nombre: String
- correo: String
- contraseña: String
- rol: String

Métodos

- registrar ()
- autenticar ()
- actualizarDatos()
- eliminar ()

- **Clase Estación**

Atributos

- id_estacion : int
- nombre: String
- dirección: String
- latitud: double
- longitud: double
- marca: String

Métodos

- registrarEstacion()
- actualizarEstacion()
- eliminarEstacion()

- **Clase Precio Combustible**

Atributos

- id_precio : int
- tipo_combustible : String
- precio: double
- fecha_actualizacion : Date

Métodos

- actualizarPrecio()
- consultarPrecio()

- Clase SimulacionViaje

Atributos

- id_simulacion : int
- origen: String
- destino: String
- distancia_km : double
- costo_estimado : double

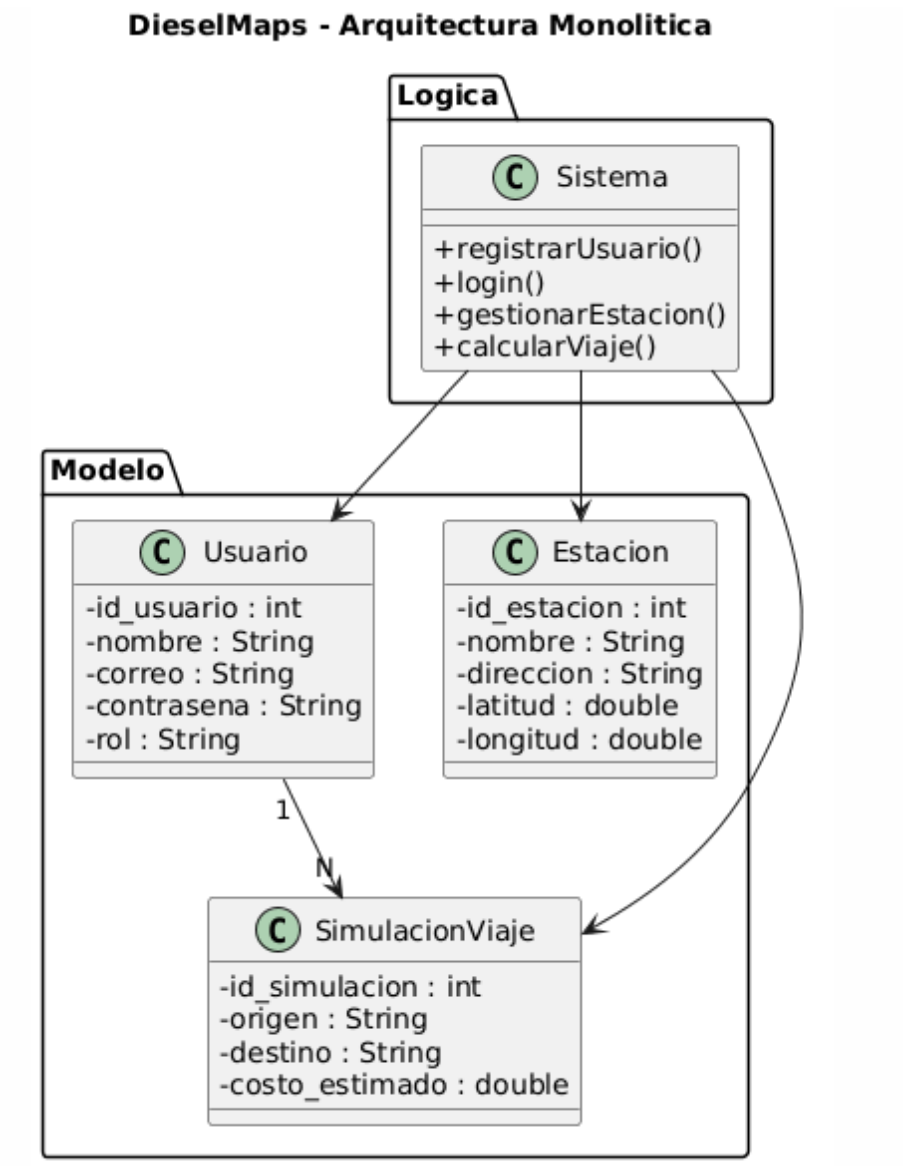
Métodos

- calcularDistancia()
- calcularCosto()
- guardarSimulacion()

- Relaciones (herencia, agregación, composición)

- Usuario 1 — N SimulacionViaje
- Estacion 1 — N PrecioCombustible
- Existe asociación entre clases.
- Sistema usa todas las clases.

- Diagrama UML de clases


DieselMaps - Arquitectura Monolitica
Logica
Sistema
+registrarUsuario()
+login()
+gestionarEstacion()
+calcularViaje()
Modelo
Usuario
-id_usuario : int
-nombre : String
-correo : String
-contrasena : String
-rol : String
Estacion
-id_estacion : int
-nombre : String
-direccion : String
-latitud : double
-longitud : double
1
N
SimulacionViaje
-id_simulacion : int
-origen : String
-destino : String
-costo_estimado : double